SW Engineering CSC648-848 Fall 2025

GatorsMatch Tutors

Team 01

Name	Role(s)
Amer Music - amusic@mail.sfsu.edu	Project Lead & Frontend Developer
Arric Sekhon	Frontend Lead
Derek Ye	Backend Lead
Rishika Bharodiya	GitHub Master
Stanley Wang	Backend Support
Alonzo Manosca	GitHub Support
Daniel Castillo	Frontend Support

Document History

	Submitted Date	Revised Date
Milestone 4	12/4/2025	

Page Index

1.) Product Summary

Product Name: GatorMatch Tutors (http://18.191.185.168/)

Product Description (Plain English, 1 paragraph)
GatorMatch Tutors is a safe, simple, SFSU-only platform that helps students quickly find credible peer tutors for the exact courses they're taking. Any @sfsu.edu student can search by course code (e.g., CSC 340), browse approved tutor listings, check availability, and send a one-round in-site message to connect—no outside email or messaging apps required. Tutor profiles highlight meaningful trust signals such as course expertise, availability, campus meeting options, and SFSU-verified eligibility. Tutors can easily create and update listings, and every listing goes through an Admin approval step to ensure accuracy and safety. What makes GatorMatch unique is its SFSU-specific design: course-centric search, campus location tags, semester-aware availability, and filters that never show empty results. The platform provides a moderated, campus-focused directory that is easy for students, tutors, administrators, and faculty to rely on.

P1 Student Functions

1. Students can register using an @sfsu.edu email address.
2. Students can sign in with an @sfsu.edu email address.
3. Students can browse and search for tutors using data-driven filters that hide options with zero matches.
4. Students can view full tutor listings including bio, subjects, courses, availability, and location tags.
5. Students can send a one-round in-site "Contact Tutor" message (no replies allowed).

P1 Tutor Functions

6. Students can enable a Tutor role to create tutor listings.
7. Tutors can create or edit their listings, including headline, bio, subjects, course prefixes, availability, and location tags.
8. Tutors can submit listings for Admin approval and see their status (Draft, Pending, Approved, Rejected).

P1 Admin Functions

9. Admins can approve or reject listings and provide rejection reasons.

10. Admins can access a role-based dashboard with an approval queue and moderation tools.

P1 System-Level Features

11. Course-centric and SFSU-specific discovery (course filters, campus locations, semester-friendly availability).

12. Role-based dashboards for Students, Tutors, and Admins.

2.) Usability Test Plan for Selected Function

2.1 Test Objectives

The objective of this usability test is to determine whether users can easily and efficiently find a suitable tutor using the Search Tutors function. We want to see how well students can locate search tools, apply filters, interpret results, and reach a tutor's detail page without confusion. This test helps us identify unclear labels, possible navigation issues, steps that cause hesitation, and how often users successfully complete the task without help. The results will show whether our search interface is intuitive for new SFSU students and whether the experience supports their goal of quickly finding academic assistance.

2.2 Test Background and Setup

This usability test will be conducted on the deployed version of GatorMatch Tutors. The system will run on a laptop or phone with a stable internet connection and at least two major browsers installed, such as Google Chrome and Safari. The test database will contain a set of realistic tutor listings across different courses so that search results behave naturally. Participants begin on the home page of the website, and no registration is required for the test. Our intended users are SFSU students who may need tutoring, since they best represent the real audience for this function. The test will take place in a quiet environment, and the facilitator will not assist unless the participant is stuck for more than three minutes. The deployment URL used for testing will be the team's public server link.

2.3 Usability Task Description (Instructions to Tester)

In this scenario, the participant acts as an SFSU student who is currently struggling with CSC 340 and wants to find a tutor who is available either on campus or on Zoom. Starting from the home page, the participant must navigate to the page that shows all tutors, use the search bar or filters to find tutors who can help with CSC 340, and then apply at least one additional filter such as subject or location. After

finding a suitable match, the participant should choose one tutor, open their detail page, and describe why they selected that tutor. Finally, the participant must identify where they would click to contact the tutor, although they do not need to send an actual message.

2.4 Plan for Evaluation of Effectiveness

To evaluate effectiveness, we will observe whether the participant can complete the full task without assistance: locating the tutor listings page, performing a course-specific search, applying additional filters correctly, opening a relevant tutor's detail page, and locating the Contact button. We will record whether each step is completed with no help, completed with minor hints, or not completed at all. We will also document errors such as selecting the wrong page or applying filters incorrectly. The overall effectiveness will be measured by the percentage of participants who successfully finish the entire task within a ten-minute time limit.

2.5 Plan for Evaluation of Efficiency

Efficiency focuses on how quickly and smoothly participants complete the scenario. We will measure the total time from the moment they start on the home page to when they identify the Contact button on the selected tutor's detail page. We will also count how many clicks and page transitions the participant makes and note whether they need to undo steps, reset filters, or navigate back. After testing multiple users, we will calculate the average completion time, fastest and slowest times, number of clicks, and identify any patterns that suggest confusing design elements.

2.6 Plan for Evaluation of User Satisfaction

After completing the task, each participant will answer a short questionnaire using a 5-point Likert scale to measure satisfaction. The questions ask whether it was easy to find a tutor, whether the search and filter options were clear, whether the information provided was useful, whether the participant would feel comfortable using this website in real life, and whether they were satisfied with the overall experience. Participants may also answer two optional open-ended questions about what they liked most and what they believe should be improved. These responses help us understand the emotional and subjective side of the user experience.

2.7 GenAI Use (Usability Test Plan)

For this section, we used a Generative AI tool to help refine the clarity and structure of our usability test plan. ChatGPT reviewed our early drafts and suggested clearer task phrasing, better separation between effectiveness and efficiency metrics, and improvements to the wording of our Likert-scale questions to ensure neutrality. We asked prompts such as "Please improve the clarity of this usability test scenario" and "Are these survey questions neutral and appropriate?" The tool was helpful for editing and organizing our ideas, so we consider the utility of GenAI in this section to be medium.

3.) QA Test Plan and QA Testing

3.1 Test Objectives

- Verify that the **Search Tutors** feature works correctly for typical and edge-case inputs.
- Confirm that search and filters return matching tutors and handle “no result” cases properly.
- Check that behavior is consistent across at least two major web browsers.

3.2 Hardware and Software Setup

- Hardware:
    - Laptop/phone with stable network connection.
- Software:
    - Operating system: Windows/macOS/Linux (any current version).
    - Browsers:
        - Browser A: Google Chrome (latest version).
        - Browser B: Safari (latest version).
    - Backend: Flask app connected to MySQL database as in project architecture.
    - Deployed server URL: http://18.191.185.168/

3.3 Feature to Be Tested

- **Browse & Search Tutors** including:
    - Free-text search by course or keyword.
    - Filters by subject, course number, location, and term/session.
    - Display of tutor listings and navigation to detail pages.

3.4 QA Test Plan Table (Executed on Two Browsers)

Test	Test Title	Test Description	Test Input	Expected Correct Output	Browser A (Chrome)	Browser B (Safari)
T1	Basic course search	Ensure searching by a valid course shows correct tutors	CSC 340	At least one tutor with CSC 340 appears	PASS	PASS
T2	Search no results	Ensure system handles empty results properly	ABC 878	"No tutors found" message appears	PASS	PASS
T3	Combined filters	Test subject + location filtering	Subject=CSC, Location=Zoom	All tutors shown match CSC + Zoom	PASS	PASS
T4	Open detail page	Check navigation to tutor details	Click first tutor	Correct tutor detail page opens	PASS	PASS

3.5 Execution Notes (Two Browsers)

We ran all four QA tests on Google Chrome and Safari. Every test worked the same on both browsers, and all results were correct. The search feature showed the right tutors, the "no results" message appeared when expected, filters worked properly, tutor detail pages opened with no issues, and long inputs did not break the page. There were no errors or layout problems in any test. All tests passed on both browsers.

3.6 GenAI Use (QA Test Plan)

Tool used

- ChatGPT

How we used the tool and benefits

- We gave the ChatGPT a short description of our search feature and asked it to suggest concrete test cases including normal, edge, and error inputs.

- It helped us think of specific cases like very long search strings and no-result scenarios, and reminded us to test across multiple browsers and handle empty states clearly.

Example prompts

- "We have a Flask web app where students search for tutors by course and filters. Suggest a QA test plan with at least 5 test cases in table format, including normal input, invalid input, and no results."

- "Please adjust these test cases so they are easy to read by non-technical stakeholders and include expected outputs."

4.) Peer Code Review (with GenAI)

4.1 Selected Code for Review

For this Milestone, our team selected the backend code supporting the **Browse & Search Tutors** feature and the **Tutor Session Request** flow. The following files were included as part of the review:

- app.py
  • Routes for tutor search, tutor profiles, session requests, dashboards

- models.py
  • SQLAlchemy models for User, Tutor, TutorApplication, Tutor, TutoringSession, Availability Blocks

- forms.py
  • Registration, login, tutor application forms, custom validators

- admin.py
  • Admin dashboard, approval/rejection flows

- db.py
  • SQLAlchemy engine and session configuration

- jitshi_helper.py
  • Helper for generating Jitsi meeting links

This code represents the backend logic responsible for finding tutors, validating SFSU-only usage, storing tutor data, handling applications, and creating tutoring sessions.

4.2 Peer Review Process (Human Reviewer)

1. **Submission of code for review**
    - The backend developer (e.g., Derek) sent a link to the relevant GitHub branch/file to another team member (e.g., Stanley) via email/Discord, requesting a formal code review.
2. **Review activities performed**
   The reviewer checked for:
    - Clear and consistent naming of variables, functions, and models (aligned with data definitions in Milestone 2).
    - Presence of header comments and helpful inline comments where logic is complex.
    - Proper handling of user input (sanitizing or parameterizing queries).
    - Separation of concerns between route logic and template rendering.
    - Readability and maintainability (avoiding deeply nested conditionals, repeated code, etc.).
3. **Feedback and communication**
    - The reviewer left comments directly in GitHub (or equivalent) pointing out:
        - Suggested renaming of some variables for clarity.
        - Opportunities to extract repeated filter conditions into helper functions.
        - Small style issues such as spacing and docstring formatting.
    - A summary of review findings was sent back via email or team chat, including any "must-fix" issues before merging.

4.3 Peer Review Using GenAI (Required)

How GenAI Was Used

We provided ChatGPT with the relevant backend files (app.py, models.py, forms.py, admin.py, jitshi_helper.py) and asked it to:

- Review program structure and clarity
- Identify potential security issues

- Suggest improvements in maintainability and refactoring
- Analyze database relationships
- Evaluate validation logic and error handling
- Highlight missing or incomplete feature logic

This helped verify architectural choices and catch inconsistencies that human reviewers may overlook.

GenAI Review Findings (Summary)

Strengths Identified by GenAI

- Good modular structure and clean ORM model definitions
- Secure password hashing and input validation
- Well-designed tutor and session models
- Safe DB usage through scoped sessions

GenAI's Suggested Improvements

1. **Implement missing search logic**
   GenAI noted that /search currently does not process query parameters, which does not align with the QA test plan.

2. **Enforce admin-only access in all admin routes**
   GenAI recommended a decorator-based solution.

3. **Replace magic strings with Enums or shared constants**
   Improves consistency and future extensibility.

4. **Improve date/time parsing reliability in session creation**
   GenAI recommended explicit error messages and standardized parsing.

5. **Add docstrings to several routes**
   Improves comprehension for future developers.

6. **Consider extracting session logic into a service layer**
   Reduces the size of route handlers and improves testability.

Utility Ranking of GenAI in This Code Review

RANK: HIGH

GenAI provided:

- Precise identification of architectural issues
- Actionable recommendations aligned with best practices
- Useful refactoring suggestions
- Security validations that reinforced our own review
- Improvements that directly help with Milestone 4 and final project quality

GenAI significantly accelerated the review process and ensured a comprehensive evaluation beyond what a single teammate could perform manually.

5.) Security Self-Check

5.1 Assets, Threats, and Mitigations

Asset	Possible Threats	Impact if Breached	Mitigation (Protection Method)
User accounts & passwords	Stolen passwords, brute-force login	Unauthorized access to user data	Store hashed passwords; limit login attempts
User profile info (name, SFSU email)	Data leaks, scraping	Privacy issues, spam	Restrict access; do not show emails publicly
Tutor listings	Unauthorized edits or deletion	Wrong or missing information	Allow edits only by tutors/admins; require admin approval
One-round messages	Spam or misuse	Harassment or abuse	Log messages; keep messages inside the site

Server & database	SQL injection, unauthorized access	Full system compromise	Use parameterized queries; secure SSH; protect environment variables

6.) Self-Check of the Adherence to Original Non-Functional Specs

#	NFR	Status	Notes
1	Use approved tools/servers (per Class CTO, M0)	DONE	
2	Optimized for standard desktop/laptop browsers (latest 2 versions of 2 major browsers)	DONE	
3	Key functions render well on mobile devices (no native app)	DONE	
4	Tutor posting & messaging limited to SFSU students	DONE	
5	Critical data stored in DB on team's deployment server	DONE	
6	No more than ~50 concurrent users expected	DONE	
7	User privacy is protected	DONE	
8	English-only UI (no localization)	DONE	
9	Application very easy and intuitive to use	DONE	
10	Follows established architecture patterns	DONE	
11	Code & repository easy to inspect and maintain	DONE	
12	Google Analytics used	DONE	
13	No email clients; only in-site messaging; single message round; no chat	DONE	
14	No payment functionality implemented or simulated	DONE	
15	Basic site security best practices applied to main data items	DONE	
16	Media formats are standard (as used in the market today)	DONE	
17	Modern SE processes & tools (collab, CI/CD,	DONE	

	GenAI, etc.) used as specified in class		
18	UI displays required disclaimer text on all pages in navbar	DONE	